

Nachname, Vorname: _____ Matrikelnummer: _____

PC-Nummer: _____ Unterschrift: _____

bearbeitete Aufgaben: ☐ 1 ☐ 2 ☐ 3 ☐ 4

von den Prüfenden auszufüllen:

☐ Bonus	☐ kein Bonus	☐ bestanden	☐ nicht bestanden
--------------------------------	-------------------------------------	------------------------------------	--

Leibniz Universität Hannover
Fachgebiet Mensch-Computer-Interaktion

Klausur Programmieren 1
23. Januar 2025, 8:00 Uhr

Gruppe 01a

Dies ist eine **90**-minütige Klausur. Tragen Sie bitte Ihren Namen, Ihre Matrikelnummer, die PC-Nummer und die bearbeiteten Aufgaben auf diesem Blatt ein und unterschreiben Sie es. Legen Sie bitte Ihren Studierendenausweis und Ihren Personalausweis zur Anwesenheitskontrolle bereit. Es sind keine zusätzlichen Hilfsmittel erlaubt. Schalten Sie bitte Ihr Mobiltelefon aus und legen Sie es weg. Für Notizen können Sie diesen Zettel verwenden. Notizen auf diesem Zettel werden nicht mitbewertet.

Wenn Sie **keinen Bonus** erlangt haben, müssen Sie von den vier gestellten Aufgaben **drei Aufgaben** erfolgreich lösen. **Mit Bonus** müssen Sie nur **zwei** der vier Aufgaben erfolgreich lösen.

Hinweise:

- Nutzen Sie das Terminal zum Kompilieren. Kommandos zum Kompilieren finden Sie in den Template-Dateien. Wechseln Sie Verzeichnisse mit `cd`. Listen Sie die Dateien mit `ls`.
- Nutzen Sie KWrite oder VisualStudioCode zum Editieren. Schalten Sie Zeilennummern ein.
- Ihre Lösungen müssen mit den jeweiligen Testfällen funktionieren und auch generelle Lösungen darstellen. Die Testfunktionen und Testfälle dürfen nicht verändert werden.
- Sie dürfen beliebige Hilfsfunktionen implementieren.

1. Implementieren Sie die Funktion `has_4_or_more_sorted_digits`. Diese Funktion soll wahr zurückgeben, wenn die übergebene Zeichenkette mindestens vier hintereinander stehende und aufsteigend sortierte Dezimalziffern enthält. Ein Beispiel wäre "xa723590c". Andernfalls soll die Funktion falsch zurückgeben. Zur Implementierung dürfen nur die Funktionen des Templates, selbst definierte Hilfsfunktionen, sowie `strlen` verwendet werden.

Template-Datei: `four_sorted_digits.c`.

2. Implementieren Sie die Funktion `Matrix transpose(Matrix m)`. Der Eingabeparameter ist eine Matrix mit `rows` Zeilen und `cols` Spalten. Die Elemente der Matrix sind als zweidimensionales `double`-Array organisiert. (C speichert zweidimensionale Arrays zeilenweise.) Die Funktion soll aus der Eingabematrix eine transponierte Ausgabematrix erzeugen, d.h. die Zeilen werden zu Spalten und umgekehrt. Die Eingabematrix darf nicht verändert werden.

Template-Datei: `transpose.c`

3. Implementieren Sie die Funktion `center_or_zero`. Diese Funktion soll die Zahl in der Mitte der Liste zurückgeben. Wenn die Liste eine gerade Anzahl an Elementen enthält, soll die Funktion 0 zurückgeben.

Template-Datei: `center_or_zero.c`

4. Tree repräsentiert einen Binärbaum mit numerischen Werten. Implementieren Sie die Funktion `is_search_tree`. Diese soll genau dann wahr zurückgeben, wenn der Baum ein Suchbaum ist. Ein Binärbaum sei genau dann ein Suchbaum, wenn für jeden Knoten gilt, dass die Werte im linken Unterbaum kleiner und die Werte im rechten Unterbaum größer sind als der Wert des Knotens.

Template-Datei: `is_search_tree.c`.